Python Programming — 100 Practice Exercises (Medium Difficulty)

The EasyLearn Academy

This set of 100 exercises follows the modules of the Programming with Python syllabus. Each exercise includes a task description, a complete working solution, and a clear explanation of how the code works. Difficulty is calibrated to **medium** — beyond pure syntax drills, but not requiring advanced libraries.

Module Map

Module	Topic	Exercises
1	Core Python Concepts	1 – 15
2	Collections (List, Tuple, Dictionary)	16 – 35
3	Functions	36 – 47
4	Modules	48 – 55
5	Input / Output & Files	56 – 65
6	Exception Handling	66 – 75
7	OOP Concepts	76 – 90
8	Database (SQL with Python)	91 – 100

MODULE 1 — Core Python Concepts

Exercise 1: Grade Calculator (if-elif-else)

Task: Take a student's marks (0–100) as input and print their grade using the standard scale: A (≥90), B (≥75), C (≥60), D (≥40), F (below 40).

```
python
```

```
marks = float(input("Enter marks (0-100): "))

if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
elif marks >= 40:
    grade = "D"
else:
    grade = "F"

print(f"Grade: {grade}")
```

Explanation: The `if-elif-else` chain is evaluated top to bottom, and the first condition that is `True` stops the chain. Ordering the checks from the highest cutoff down to the lowest ensures each mark lands in exactly one bracket. Using `float()` allows decimal marks like 89.5.

Exercise 2: Nested If-Else — Triangle Classifier

Task: Given three side lengths, determine whether they form a valid triangle, and if so, whether it is equilateral, isosceles, or scalene.

python

```
a = float(input("Side a: "))
b = float(input("Side b: "))
c = float(input("Side c: "))

if a + b > c and b + c > a and a + c > b:
    if a == b == c:
        print("Equilateral triangle")
    else:
        if a == b or b == c or a == c:
            print("Isosceles triangle")
        else:
            print("Scalene triangle")
else:
    print("Not a valid triangle")
```

Explanation: The outer `if` applies the triangle inequality rule (each side must be shorter than the sum of the other two). Only when that holds do the nested `if-else` blocks classify the triangle by comparing side equality, showing how nested conditionals narrow down a

decision step by step.

Exercise 3: Multiplication Table (for loop)

Task: Print the multiplication table of a number entered by the user, from 1 to 10.

```python
n = int(input("Enter a number: "))

for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

Explanation: `range(1, 11)` generates integers 1 through 10. On each iteration, `i` takes the next value, and an f-string formats the equation cleanly without manual string concatenation.

Exercise 4: Digital Root (while loop)

Task: Repeatedly sum the digits of a number until a single digit remains (e.g., 9875 → 29 → 11 → 2).

```python
n = int(input("Enter a number: "))

while n >= 10:
    total = 0
    while n > 0:
        total += n % 10
        n //= 10
    n = total

print(f"Digital root: {n}")
```

Explanation: The inner `while` loop peels off digits using `% 10` (last digit) and `// 10` (remove last digit), accumulating their sum. The outer loop repeats this process as long as the result still has more than one digit, demonstrating nested `while` loops working together.

Exercise 5: Star Pyramid Pattern (nested loops)

Task: Print a right-aligned pyramid of stars, n rows tall.

```python
```

```python
n = int(input("Enter number of rows: "))

for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

Explanation: For row `i`, the number of leading spaces (`n - i`) shrinks as the row number grows, while the number of stars (`2*i - 1`) grows by 2 each row. String multiplication (`" " * count`) builds each row without a second inner loop.

Exercise 6: First Prime in Range (break)

Task: Find and print the first prime number greater than or equal to a given number.

python

```python
n = int(input("Enter a starting number: "))

candidate = n
while True:
    is_prime = candidate > 1
    for i in range(2, int(candidate ** 0.5) + 1):
        if candidate % i == 0:
            is_prime = False
            break
    if is_prime:
        print(f"First prime >= {n} is {candidate}")
        break
    candidate += 1
```

Explanation: The inner `for` loop tests divisibility only up to the square root of the candidate (a standard efficiency trick) and uses `break` to exit immediately once a divisor is found — there's no need to keep checking. The outer `while True` loop increments the candidate until `is_prime` is confirmed `True`, then `break`s out entirely.

Exercise 7: Skip Multiples of 5 (continue)

Task: Print all odd numbers from 1 to 50, but skip any that are also multiples of 5.

python

```python
for num in range(1, 51):
    if num % 2 == 0:
        continue
    if num % 5 == 0:
        continue
    print(num, end=" ")
```

Explanation: `continue` immediately jumps to the next loop iteration, skipping the `print` statement for that value. Two separate checks keep the logic readable: one filters out even numbers, the other filters out multiples of 5.

Exercise 8: Menu Skeleton (pass)

Task: Build a menu-driven program skeleton with options for Add, Update, Delete, and Exit, where unimplemented options use `pass` as a placeholder.

python

```python
while True:
    print("\n1. Add Record\n2. Update Record\n3. Delete Record\n4. Exit")
    choice = input("Enter choice: ")

    if choice == "1":
        pass  # TODO: implement add logic
    elif choice == "2":
        pass  # TODO: implement update logic
    elif choice == "3":
        pass  # TODO: implement delete logic
    elif choice == "4":
        print("Exiting program.")
        break
    else:
        print("Invalid choice, try again.")
```

Explanation: `pass` is a no-op statement that lets a code block exist syntactically without doing anything yet — useful when sketching a program's structure before filling in real logic. The loop keeps showing the menu until the user selects "Exit", which uses `break` to stop the loop.

Exercise 9: Palindrome Checker (string slicing)

Task: Check whether a given string reads the same forwards and backwards, ignoring case and spaces.

python

```python
text = input("Enter a string: ")
cleaned = text.lower().replace(" ", "")

if cleaned == cleaned[::-1]:
    print("Palindrome")
else:
    print("Not a palindrome")
```

Explanation: `[::-1]` is slice notation with a step of -1, which reverses a string without a loop. Normalizing the string first (lowercasing and removing spaces) ensures phrases like "Was it a car or a cat I saw" are compared fairly.

Exercise 10: Word Frequency Counter (string methods)

Task: Count how many times each word appears in a sentence.

```python
sentence = input("Enter a sentence: ").lower()
words = sentence.split()

frequency = {}
for word in words:
    frequency[word] = frequency.get(word, 0) + 1

for word, count in frequency.items():
    print(f"{word}: {count}")
```

Explanation: `.split()` breaks the sentence into a list of words on whitespace. `dict.get(word, 0)` returns the current count (or 0 if the word hasn't been seen), so `frequency[word] = frequency.get(word, 0) + 1` safely increments the count for both new and repeated words.

Exercise 11: Caesar Cipher (string manipulation)

Task: Encrypt a message by shifting each letter forward by a fixed number of positions in the alphabet.

```python
```

```python
text = input("Enter text: ")
shift = int(input("Enter shift value: "))

result = ""
for char in text:
    if char.isalpha():
        base = ord('A') if char.isupper() else ord('a')
        result += chr((ord(char) - base + shift) % 26 + base)
    else:
        result += char

print(f"Encrypted: {result}")
```

Explanation: `ord()` converts a character to its numeric code, and `chr()` converts it back. Subtracting `base` shifts the letter into a 0–25 range, `% 26` wraps around at the end of the alphabet (so 'z' shifted by 1 becomes 'a'), and adding `base` back converts it to a character again. Non-letter characters (spaces, punctuation) pass through unchanged.

Exercise 12: Vowel Counter (accessing strings)

Task: Count the number of vowels in a string and report each vowel's individual count.

```python
text = input("Enter a string: ").lower()
vowels = "aeiou"
counts = {v: 0 for v in vowels}

for char in text:
    if char in vowels:
        counts[char] += 1

print("Vowel counts:", counts)
print("Total vowels:", sum(counts.values()))
```

Explanation: A dictionary comprehension `{v: 0 for v in vowels}` initializes a counter for each vowel. Looping character by character and checking membership with `in` is a direct way to "access" and inspect a string one element at a time, as emphasized in this module.

Exercise 13: Title Case Without .title() (basic string operations)

Task: Capitalize the first letter of every word in a sentence without using the built-in `.title()` method.

```python
```

```
sentence = input("Enter a sentence: ")
words = sentence.split()

capitalized = []
for word in words:
    capitalized.append(word[0].upper() + word[1:].lower())

print(" ".join(capitalized))
```

Explanation: For each word, slicing separates the first character (`word[0]`, uppercased) from the rest (`word[1:]`, lowercased), and concatenation joins them back together. `" ".join(list)` reassembles the words into a sentence — the counterpart operation to `.split()`.

Exercise 14: Simple Calculator (functions & methods intro via operators)

Task: Build a calculator that performs addition, subtraction, multiplication, or division based on user choice, with divide-by-zero protection.

python

```
a = float(input("Enter first number: "))
op = input("Enter operator (+, -, *, /): ")
b = float(input("Enter second number: "))

if op == "+":
    print(f"Result: {a + b}")
elif op == "-":
    print(f"Result: {a - b}")
elif op == "*":
    print(f"Result: {a * b}")
elif op == "/":
    if b != 0:
        print(f"Result: {a / b}")
    else:
        print("Error: Division by zero")
else:
    print("Invalid operator")
```

Explanation: The program reads the operator as a string and matches it against known symbols with `elif`. Division is guarded with an explicit zero check before performing `a / b`, avoiding a runtime crash — good practice before formal exception handling (covered in Module 6).

Exercise 15: String Slice Analyzer

Task: Given a string, print its first half, second half, every alternate character, and the string reversed — all using slicing.

```python
text = input("Enter a string: ")
mid = len(text) // 2

print("First half:", text[:mid])
print("Second half:", text[mid:])
print("Every alternate char:", text[::2])
print("Reversed:", text[::-1])
```

Explanation: Slice syntax `text[start:stop:step]` is flexible: omitting `start`/`stop` defaults to the beginning/end, and `step=2` selects every second character. This exercise consolidates the slicing patterns used throughout the module into one reference program.

MODULE 2 — Collections (List, Tuple, Dictionary)

Exercise 16: Remove Duplicates, Preserve Order (List)

Task: Remove duplicate values from a list while keeping the original order of first appearance.

```python
numbers = [4, 2, 5, 2, 3, 4, 1, 5]

seen = set()
unique = []
for n in numbers:
    if n not in seen:
        seen.add(n)
        unique.append(n)

print(unique)
```

Explanation: A `set` gives O(1) membership checks, so `n not in seen` is fast even for long lists. Only values not already recorded are appended to `unique`, which preserves the original left-to-right order — something `list(set(numbers))` would not guarantee, since sets are unordered.

Exercise 17: Second Largest Number (List)

Task: Find the second largest number in a list without sorting the entire list.

python

```python
numbers = [12, 45, 2, 41, 45, 9]

largest = second = float('-inf')
for n in numbers:
    if n > largest:
        second = largest
        largest = n
    elif n > second and n != largest:
        second = n

print(f"Largest: {largest}, Second largest: {second}")
```

Explanation: Two trackers, `largest` and `second`, are updated in a single pass. Whenever a new maximum is found, the old maximum "demotes" to `second` before `largest` is replaced. This is more efficient than sorting (O(n) vs O(n log n)) and reinforces manual list traversal.

Exercise 18: Rotate List Left by N (List slicing)

Task: Rotate the elements of a list to the left by `n` positions.

python

```python
lst = [1, 2, 3, 4, 5, 6, 7]
n = 3

rotated = lst[n:] + lst[:n]
print(rotated)
```

Explanation: Slicing splits the list into two parts: everything from index `n` onward, and everything before it. Concatenating them in swapped order (`lst[n:] + lst[:n]`) shifts the first `n` elements to the end, achieving a left rotation in one line.

Exercise 19: Bubble Sort Two Merged Lists (List methods)

Task: Merge two lists and sort the combined list using the bubble sort algorithm (without calling `sort()` or `sorted()`).

python

```python
list1 = [5, 1, 9]
list2 = [3, 8, 2]

merged = list1 + list2
n = len(merged)

for i in range(n):
    for j in range(0, n - i - 1):
        if merged[j] > merged[j + 1]:
            merged[j], merged[j + 1] = merged[j + 1], merged[j]

print(merged)
```

Explanation: The `+` operator concatenates the two lists. Bubble sort then repeatedly compares adjacent elements and swaps them if out of order; each full pass "bubbles" the largest remaining value to its correct position, and the `- i - 1` bound skips the already-sorted tail on later passes.

Exercise 20: Squares of Even Numbers (List comprehension)

Task: Given a list of numbers, build a new list containing the squares of only the even numbers.

python

```python
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
squares = [n ** 2 for n in numbers if n % 2 == 0]
print(squares)
```

Explanation: A list comprehension combines a `for` loop, a filter condition, and an expression in one readable line: `[expression for item in iterable if condition]`. Here it filters even numbers first (`if n % 2 == 0`), then squares each surviving value.

Exercise 21: Matrix Transpose (Nested list)

Task: Transpose a matrix represented as a list of lists (rows become columns).

python

```python
matrix = [[1, 2, 3],
          [4, 5, 6]]

transposed = [[row[i] for row in matrix] for i in range(len(matrix[0]))]

for row in transposed:
    print(row)
```

Explanation: The outer comprehension iterates over each column index `i`; the inner comprehension picks the `i`-th element from every row. Together they rebuild the matrix with rows and columns swapped, a common pattern for working with nested lists.

Exercise 22: Common Elements Between Two Lists (List function)

Task: Find all elements that appear in both of two given lists, without duplicates.

```python
list1 = [1, 2, 3, 4, 5]
list2 = [3, 4, 5, 6, 7]

common = list(set(list1) & set(list2))
print(f"Common elements: {common}")
```

Explanation: Converting both lists to `set`s allows the `&` operator to compute their intersection directly — far simpler than nested loops with manual comparisons. Wrapping the result back in `list()` gives a plain list output.

Exercise 23: Swap Two Variables (Tuple unpacking)

Task: Swap the values of two variables without using a temporary variable.

```python
a = 10
b = 25

a, b = b, a
print(f"a = {a}, b = {b}")
```

Explanation: The right-hand side `b, a` first builds a temporary tuple `(b, a)`, which is then unpacked into `a` and `b` on the left. This single-line pattern is a distinctive Python feature made possible by tuples and simultaneous assignment.

Exercise 24: Student Record Search (Tuple operations)

Task: Store student records as tuples `(name, age, grade)` in a list, then search for a student by name.

```python
students = [("Aarav", 20, "A"), ("Diya", 21, "B"), ("Kabir", 19, "A")]

search_name = input("Enter student name to search: ")
found = False
for name, age, grade in students:
    if name.lower() == search_name.lower():
        print(f"Name: {name}, Age: {age}, Grade: {grade}")
        found = True
        break

if not found:
    print("Student not found.")
```

Explanation: Each tuple is unpacked directly in the `for` loop header (`for name, age, grade in students`), avoiding manual indexing like `student[0]`. This makes the code self-documenting since each field has a meaningful name.

Exercise 25: Tuple Immutability Demonstration

Task: Show that tuples cannot be modified directly, and demonstrate the workaround of converting to a list, modifying, and converting back.

```python
coordinates = (10, 20, 30)

try:
    coordinates[0] = 99
except TypeError as e:
    print(f"Error: {e}")

temp_list = list(coordinates)
temp_list[0] = 99
coordinates = tuple(temp_list)
print(f"Updated tuple: {coordinates}")
```

Explanation: Tuples are immutable, so direct item assignment raises a `TypeError`, caught here to show the error without crashing. The standard workaround — `list()` to convert, modify freely, then `tuple()` to convert back — is a common pattern when a tuple's contents need to change.

Exercise 26: Distance Between Two Points (Nested tuples)

Task: Store two 2D points as nested tuples and calculate the Euclidean distance between them.

```python
points = ((3, 4), (7, 1))

(x1, y1), (x2, y2) = points
distance = ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5

print(f"Distance: {distance:.2f}")
```

Explanation: Nested tuple unpacking `(x1, y1), (x2, y2) = points` extracts all four coordinates in one step. The formula then applies the standard distance equation, and `:.2f` formats the result to two decimal places.

Exercise 27: Most Frequent Element (Tuple methods)

Task: Find the most frequently occurring element in a tuple using the `.count()` method.

```python
data = (1, 3, 2, 3, 4, 3, 2, 5)

most_common = max(set(data), key=data.count)
print(f"Most frequent element: {most_common} (appears {data.count(most_common)}
```

Explanation: `set(data)` gives each unique value once, and `max(..., key=data.count)` evaluates `data.count(x)` for every unique `x` to find which one occurs most often. This directly exercises the tuple's built-in `.count()` method rather than manually tallying frequencies.

Exercise 28: Min-Max Finder (Tuple as function return)

Task: Write a function that returns both the minimum and maximum of a list as a tuple.

```python
def min_max(numbers):
    return min(numbers), max(numbers)

values = [23, 5, 67, 12, 89, 1]
lowest, highest = min_max(values)
print(f"Min: {lowest}, Max: {highest}")
```

Explanation: Writing `return min(numbers), max(numbers)` automatically packs both values into a tuple. On the caller's side, `lowest, highest = min_max(values)` unpacks that tuple back into two named variables — a clean way for a function to return more than one value.

Exercise 29: Word Count Using a Dictionary

Task: Count occurrences of each character in a string using a dictionary.

```python
text = input("Enter text: ")
char_count = {}

for char in text:
    if char != " ":
        char_count[char] = char_count.get(char, 0) + 1

for char, count in sorted(char_count.items()):
    print(f"'{char}': {count}")
```

Explanation: Similar to the earlier word-frequency exercise, `.get(char, 0)` avoids a `KeyError` for characters seen for the first time. `sorted(char_count.items())` orders the output alphabetically by key before printing.

Exercise 30: Student-Grade Lookup with .get()

Task: Build a dictionary of student grades and safely look up a student, showing a default message if not found.

```python
grades = {"Aarav": "A", "Diya": "B", "Kabir": "A"}

name = input("Enter a student name: ")
result = grades.get(name, "Student not found")
print(result)
```

Explanation: `dict.get(key, default)` returns the value for `key` if it exists, or the given default otherwise — without raising an exception the way `grades[name]` would for a missing key. This is the safe, idiomatic way to perform optional dictionary lookups.

Exercise 31: Merge Two Dictionaries with Conflict Resolution

Task: Merge two dictionaries of product prices; when a product exists in both, keep the higher price.

python

```python
shop1 = {"pen": 10, "book": 50, "bag": 300}
shop2 = {"pen": 12, "book": 45, "pencil": 5}

merged = shop1.copy()
for item, price in shop2.items():
    if item in merged:
        merged[item] = max(merged[item], price)
    else:
        merged[item] = price

print(merged)
```

Explanation: `.copy()` creates a fresh dictionary so `shop1` isn't mutated. Looping through `shop2.items()` and checking `if item in merged` lets the program apply custom conflict-resolution logic (`max`) instead of simply letting the second dictionary silently overwrite the first, as `{**shop1, **shop2}` would.

Exercise 32: Inventory Management (Dict methods)

Task: Build a simple inventory system supporting add, update quantity, and remove operations on a dictionary.

python

```python
inventory = {"apples": 50, "bananas": 30}

inventory["oranges"] = 20          # add
inventory["apples"] += 10          # update
inventory.pop("bananas", None)     # remove safely

for item, qty in inventory.items():
    print(f"{item}: {qty}")
```

Explanation: Assigning to a new key adds an entry; assigning to an existing key updates it. `dict.pop(key, None)` removes a key and returns its value, but won't raise an error if the key is already missing — safer than `del inventory[key]` for this use case.

Exercise 33: Employee Database (Nested dict)

Task: Store employee records in a dictionary of dictionaries and print each employee's details.

python

```python
employees = {
    "E101": {"name": "Riya", "department": "IT", "salary": 55000},
    "E102": {"name": "Sam", "department": "HR", "salary": 48000},
}

for emp_id, details in employees.items():
    print(f"{emp_id}: {details['name']} works in {details['department']}, earns
```

Explanation: Each employee ID maps to an inner dictionary of that employee's fields, forming a nested structure. `details['name']` accesses a value inside the inner dictionary, showing how to navigate two levels of key-based lookup.

Exercise 34: Invert a Dictionary (Dict comprehension)

Task: Swap the keys and values of a dictionary using a dictionary comprehension.

```python
original = {"a": 1, "b": 2, "c": 3}
inverted = {value: key for key, value in original.items()}

print(inverted)
```

Explanation: `{value: key for key, value in original.items()}` iterates over each key-value pair and rebuilds the dictionary with the roles reversed. This only works cleanly when the original values are unique (and hashable), since duplicate values would overwrite each other as keys.

Exercise 35: Group Words by First Letter (Dict + List)

Task: Given a list of words, group them into a dictionary keyed by their first letter.

```python
words = ["apple", "banana", "avocado", "cherry", "blueberry", "apricot"]
groups = {}

for word in words:
    first_letter = word[0]
    groups.setdefault(first_letter, []).append(word)

for letter, word_list in sorted(groups.items()):
    print(f"{letter}: {word_list}")
```

Explanation: `dict.setdefault(key, [])` returns the list already stored at `key`, or creates

and returns a new empty list if the key doesn't exist yet — either way, `.append(word)` then adds to it in the same line. This avoids writing a separate "if key not in groups" check.

MODULE 3 — Functions

Exercise 36: Simple Interest Calculator (Defining/calling a function)

Task: Write a function that calculates simple interest given principal, rate, and time.

python

```python
def simple_interest(principal, rate, time):
    return (principal * rate * time) / 100

p = float(input("Principal: "))
r = float(input("Rate (%): "))
t = float(input("Time (years): "))

print(f"Simple Interest: {simple_interest(p, r, t)}")
```

Explanation: `def simple_interest(principal, rate, time):` defines a reusable block of logic that takes three inputs and returns one output. Separating the calculation into a function (rather than inline code) makes it easy to reuse and test independently of the input-gathering code.

Exercise 37: Greeting with Default Arguments

Task: Write a function that greets a person by name, defaulting to English but supporting other languages.

python

```python
def greet(name, language="English"):
    greetings = {"English": "Hello", "Hindi": "Namaste", "Gujarati": "Kem cho"}
    word = greetings.get(language, "Hello")
    print(f"{word}, {name}!")

greet("Aarav")
greet("Riya", "Gujarati")
```

Explanation: `language="English"` gives the parameter a default value, so calling `greet("Aarav")` works without a second argument. Passing an explicit value, as in `greet("Riya", "Gujarati")`, overrides the default — demonstrating how default arguments make functions flexible without forcing every caller to specify every parameter.

Exercise 38: Sum of Variable Numbers (*args)

Task: Write a function that accepts any number of numeric arguments and returns their sum.

```python
def total_sum(*numbers):
    return sum(numbers)

print(total_sum(1, 2, 3))
print(total_sum(10, 20, 30, 40, 50))
```

Explanation: `*numbers` collects any number of positional arguments into a tuple called `numbers`, so the function works whether it's called with 2 arguments or 20. Python's built-in `sum()` then adds up that tuple's contents.

Exercise 39: Print Student Profile (**kwargs)

Task: Write a function that accepts any number of keyword arguments and prints them as a profile.

```python
def student_profile(**details):
    for key, value in details.items():
        print(f"{key}: {value}")

student_profile(name="Kabir", age=20, course="Python", city="Bhavnagar")
```

Explanation: `**details` collects keyword arguments into a dictionary, letting the caller supply any combination of named fields without the function needing to declare each one in advance. This is useful for functions where the exact set of fields may vary between calls.

Exercise 40: Min, Max, and Average of a List (Return multiple values)

Task: Write a function that returns the minimum, maximum, and average of a list of numbers.

```python
def analyze(numbers):
    return min(numbers), max(numbers), sum(numbers) / len(numbers)

values = [23, 45, 12, 67, 34]
lowest, highest, avg = analyze(values)
print(f"Min: {lowest}, Max: {highest}, Average: {avg:.2f}")
```

Explanation: Returning `min(numbers), max(numbers), sum(numbers)/len(numbers)` packs three results into one tuple, which is unpacked into three variables at the call site. This keeps related calculations grouped in a single function call rather than three separate ones.

Exercise 41: Factorial (Recursive function)

Task: Write a recursive function to compute the factorial of a number.

```python
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

num = int(input("Enter a number: "))
print(f"{num}! = {factorial(num)}")
```

Explanation: The function calls itself with a smaller input (`n - 1`) on each step, moving toward the **base case** (`n <= 1`, which returns 1 without further recursion). Each call multiplies its own `n` by the result of the smaller subproblem, building up the full factorial as the calls return.

Exercise 42: Fibonacci Series (Recursive function)

Task: Print the first `n` terms of the Fibonacci series using recursion.

```python
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

count = int(input("How many terms? "))
for i in range(count):
    print(fibonacci(i), end=" ")
```

Explanation: Each Fibonacci number is the sum of the two before it, so `fibonacci(n)` recursively calls itself for `n-1` and `n-2` until it hits the base cases `fibonacci(0) = 0` or `fibonacci(1) = 1`. The outer loop simply calls this function once per term to print the full sequence.

Exercise 43: Sort Tuples by Second Element (Lambda)

Task: Sort a list of (name, score) tuples by score in descending order using a lambda

function.

```python
students = [("Aarav", 82), ("Diya", 95), ("Kabir", 78)]

students.sort(key=lambda student: student[1], reverse=True)
print(students)
```

Explanation: `sort()`'s `key` parameter takes a function that extracts the value to sort by from each element; a `lambda` provides that function inline without a separate `def`. `student[1]` picks the score out of each tuple, and `reverse=True` sorts from highest to lowest.

Exercise 44: Celsius to Fahrenheit (Lambda + map)

Task: Convert a list of Celsius temperatures to Fahrenheit using `map()` and a lambda.

```python
celsius = [0, 20, 37, 100]
fahrenheit = list(map(lambda c: (c * 9/5) + 32, celsius))

print(fahrenheit)
```

Explanation: `map(function, iterable)` applies the given function to every element of the iterable and returns an iterator of results. Here the lambda expresses the conversion formula inline, and `list()` collects the mapped results into a regular list for display.

Exercise 45: Filter Prime Numbers (Lambda + filter)

Task: Given a list of numbers, use `filter()` and a helper function to extract only the prime numbers.

```python
```

```python
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

numbers = list(range(2, 30))
primes = list(filter(lambda n: is_prime(n), numbers))
print(primes)
```

Explanation: `filter(function, iterable)` keeps only the elements for which `function` returns `True`. The lambda wraps the `is_prime` helper so it can be passed as the filtering condition, and `list()` materializes the filtered result.

Exercise 46: Counter Using Global Keyword

Task: Write a function that increments a global counter each time it's called, and demonstrate the difference from a local variable.

python

```python
count = 0

def increment():
    global count
    count += 1
    local_message = "This variable only exists inside the function"
    print(local_message)

for _ in range(3):
    increment()

print(f"Final count: {count}")
```

Explanation: Without `global count`, the line `count += 1` would create a new local variable instead of modifying the outer one, causing an error since `count` would be referenced before assignment locally. The `global` keyword tells Python to use the outer-scope variable. `local_message`, by contrast, exists only during each function call and disappears afterward.

Exercise 47: Apply Custom Operation to List (Higher-order function)

Task: Write a function that takes another function as a parameter and applies it to every

element of a list.

```python
def apply_operation(numbers, operation):
    return [operation(n) for n in numbers]

def square(x):
    return x * x

def cube(x):
    return x ** 3

values = [1, 2, 3, 4]
print(apply_operation(values, square))
print(apply_operation(values, cube))
```

Explanation: Because functions are first-class objects in Python, `apply_operation` can accept `square` or `cube` as an argument (`operation`) and call `operation(n)` inside its list comprehension. This makes `apply_operation` reusable for any transformation, not just one hard-coded formula.

MODULE 4 — Modules

Exercise 48: Circle Calculator (math module)

Task: Calculate the area and circumference of a circle using the `math` module's value of pi.

```python
import math

radius = float(input("Enter radius: "))
area = math.pi * radius ** 2
circumference = 2 * math.pi * radius

print(f"Area: {area:.2f}")
print(f"Circumference: {circumference:.2f}")
```

Explanation: `import math` brings in the standard `math` module, which provides `math.pi` as a precise constant instead of typing `3.14` manually. Using the module keeps the calculation accurate and communicates intent clearly to anyone reading the code.

Exercise 49: Quadratic Equation Solver (math module)

Task: Solve `ax² + bx + c = 0` for real roots using the quadratic formula.

python

```python
import math

a = float(input("Enter a: "))
b = float(input("Enter b: "))
c = float(input("Enter c: "))

discriminant = b ** 2 - 4 * a * c

if discriminant > 0:
    root1 = (-b + math.sqrt(discriminant)) / (2 * a)
    root2 = (-b - math.sqrt(discriminant)) / (2 * a)
    print(f"Two real roots: {root1:.2f}, {root2:.2f}")
elif discriminant == 0:
    root = -b / (2 * a)
    print(f"One real root: {root:.2f}")
else:
    print("No real roots (complex roots exist)")
```

Explanation: `math.sqrt()` computes the square root needed by the quadratic formula. The discriminant (`b² - 4ac`) is checked first to decide which case applies — two roots, one repeated root, or no real roots — avoiding a `ValueError` from taking the square root of a negative number.

Exercise 50: Dice Roll Simulator (random module)

Task: Simulate rolling a six-sided die a given number of times and show the results.

python

```python
import random

rolls = int(input("How many times to roll? "))
results = [random.randint(1, 6) for _ in range(rolls)]

print("Rolls:", results)
print(f"Average: {sum(results) / len(results):.2f}")
```

Explanation: `random.randint(1, 6)` returns a random integer between 1 and 6 inclusive, simulating one die roll. The list comprehension repeats this the requested number of times, and the results are then summarized.

Exercise 51: Random Password Generator (random module)

Task: Generate a random password of a given length using letters, digits, and symbols.

python

```python
import random
import string

length = int(input("Enter password length: "))
characters = string.ascii_letters + string.digits + "!@#$%"

password = "".join(random.choice(characters) for _ in range(length))
print(f"Generated password: {password}")
```

Explanation: `string.ascii_letters` and `string.digits` provide ready-made character sets instead of typing them manually. `random.choice(characters)` picks one random character from that pool, and the generator expression repeats this `length` times, joined into a single string.

Exercise 52: Custom Module — Temperature Utilities

Task: Create a separate module `temp_utils.py` with conversion functions, then import and use it in a main script.

python

```python
# --- File: temp_utils.py ---
def celsius_to_fahrenheit(c):
    return (c * 9/5) + 32

def fahrenheit_to_celsius(f):
    return (f - 32) * 5/9
```

python

```python
# --- File: main.py ---
import temp_utils

print(temp_utils.celsius_to_fahrenheit(25))
print(temp_utils.fahrenheit_to_celsius(98.6))
```

Explanation: Any `.py` file can act as a module; saving related functions in `temp_utils.py` and using `import temp_utils` in another file makes those functions accessible via `temp_utils.function_name()`. This is how large programs are organized into manageable, reusable pieces instead of one long script.

Exercise 53: File Path Operations (Packages — os.path)

Task: Use the `os.path` sub-module (part of the `os` package) to inspect a file path's components.

```python
import os

path = "/home/user/documents/report.pdf"

print("Directory:", os.path.dirname(path))
print("Filename:", os.path.basename(path))
print("Extension:", os.path.splitext(path)[1])
print("Exists on this system:", os.path.exists(path))
```

Explanation: `os.path` is a sub-module inside the `os` package, accessed with dotted notation (`os.path.dirname`). This illustrates how packages group related modules together, and each function here extracts a different piece of information from the same path string.

Exercise 54: Age Calculator (datetime module)

Task: Calculate a person's age in years given their birth year, using the `datetime` module to get the current year.

```python
import datetime

birth_year = int(input("Enter your birth year: "))
current_year = datetime.datetime.now().year

age = current_year - birth_year
print(f"You are {age} years old (approximately).")
```

Explanation: `datetime.datetime.now()` returns the current date and time; `.year` extracts just the year component. Using this instead of a hard-coded year keeps the program accurate no matter when it is run.

Exercise 55: Dice Statistics (Composition — random + math + statistics)

Task: Roll a die many times and calculate the average and standard deviation of the results, combining multiple modules.

```python
```

```python
import random
import statistics

rolls = [random.randint(1, 6) for _ in range(1000)]

mean = statistics.mean(rolls)
std_dev = statistics.stdev(rolls)

print(f"Mean: {mean:.2f}")
print(f"Standard Deviation: {std_dev:.2f}")
```

Explanation: This program **composes** two modules together: `random` generates the raw data, and `statistics` analyzes it. Composition — building a solution out of smaller pieces from different modules — is a core idea in this unit, since it avoids reimplementing statistical formulas by hand.

MODULE 5 — Input / Output & Files

Exercise 56: Formatted Marksheet (Printing on screen)

Task: Print a neatly formatted student marksheet using f-strings with aligned columns.

python

```python
name = "Riya Shah"
subjects = {"Math": 88, "Science": 92, "English": 79}

print(f"{'Marksheet for ' + name:-^40}")
for subject, marks in subjects.items():
    print(f"{subject:<15}{marks:>10}")
print(f"{'Total':<15}{sum(subjects.values()):>10}")
```

Explanation: Format specifiers inside f-strings control alignment and width: `{subject:<15}` left-aligns text in a 15-character field, `{marks:>10}` right-aligns numbers in a 10-character field, and `{'text':-^40}` centers text padded with dashes. Together they produce a clean, table-like layout without a formatting library.

Exercise 57: BMI Calculator (Reading from keyboard)

Task: Read a person's weight and height from the keyboard and calculate their Body Mass Index.

python

```python
weight = float(input("Enter weight in kg: "))
height = float(input("Enter height in meters: "))

bmi = weight / (height ** 2)
print(f"Your BMI is {bmi:.2f}")

if bmi < 18.5:
    print("Category: Underweight")
elif bmi < 25:
    print("Category: Normal")
elif bmi < 30:
    print("Category: Overweight")
else:
    print("Category: Obese")
```

Explanation: `input()` always returns a string, so `float()` converts the keyboard entry into a usable number before it's used in the BMI formula (`weight / height²`). The result is then classified into a standard health category using a simple `if-elif` chain.

Exercise 58: Sum of N Numbers (Reading multiple inputs)

Task: Ask the user how many numbers they want to enter, then read each one and print the total.

python

```python
count = int(input("How many numbers? "))
total = 0

for i in range(count):
    num = float(input(f"Enter number {i + 1}: "))
    total += num

print(f"Sum: {total}")
```

Explanation: The first input determines how many times the loop should run, and each iteration reads one more value with a dynamically numbered prompt (`f"Enter number {i + 1}: "`). This is a common pattern for reading a variable amount of keyboard input.

Exercise 59: Write Names to a File (File writing)

Task: Write a list of student names to a text file, one name per line.

python

```
names = ["Aarav", "Diya", "Kabir", "Riya"]

file = open("students.txt", "w")
for name in names:
    file.write(name + "\n")
file.close()

print("Names written to students.txt")
```

Explanation: `open("students.txt", "w")` opens (or creates) a file in write mode, which overwrites any existing content. `.write()` adds text but does not add newlines automatically, so `"\n"` is appended manually to put each name on its own line. `.close()` releases the file so changes are properly saved.

Exercise 60: Count Lines in a File (File reading)

Task: Read a text file line by line and report how many lines it contains.

python

```
with open("students.txt", "r") as file:
    lines = file.readlines()

print(f"Total lines: {len(lines)}")
for i, line in enumerate(lines, start=1):
    print(f"{i}: {line.strip()}")
```

Explanation: The `with` statement (a context manager) opens the file and automatically closes it afterward, even if an error occurs — safer than manually calling `.close()`. `.readlines()` returns a list of lines, and `.strip()` removes the trailing newline character when printing each one.

Exercise 61: Timestamped Log Entries (File append)

Task: Append a new log entry with the current timestamp to a log file each time the program runs.

python

```python
import datetime

message = input("Enter log message: ")
timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")

with open("log.txt", "a") as file:
    file.write(f"[{timestamp}] {message}\n")

print("Log entry saved.")
```

Explanation: Mode `"a"` (append) adds new content to the end of the file instead of overwriting it, which is essential for a running log. `strftime()` formats the current date and time into a readable string that gets prefixed to every entry.

Exercise 62: Copy File Contents (File read/write)

Task: Copy the contents of one text file into another.

```python
with open("students.txt", "r") as source:
    content = source.read()

with open("students_backup.txt", "w") as destination:
    destination.write(content)

print("File copied successfully.")
```

Explanation: `.read()` loads the entire source file into a single string. Two separate `with` blocks are used because one file is being read while a different one is being written — each `with` block automatically closes its own file when it finishes.

Exercise 63: Word Count in a File

Task: Count the total number of words in a text file.

```python
with open("students.txt", "r") as file:
    content = file.read()

word_count = len(content.split())
print(f"Total words: {word_count}")
```

Explanation: `.split()` with no arguments splits on any whitespace (spaces, tabs,

newlines), so it correctly counts words regardless of how the file's lines are separated. `len()` on the resulting list gives the word count.

Exercise 64: Write Student Data to CSV

Task: Write a list of student records to a CSV file using the `csv` module.

```python
import csv

students = [["Name", "Age", "Grade"], ["Aarav", 20, "A"], ["Diya", 21, "B"]]

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerows(students)

print("Data written to students.csv")
```

Explanation: The `csv` module handles the details of properly formatting comma-separated values (like quoting fields that contain commas), which is more reliable than writing commas manually with `.write()`. `newline=""` prevents extra blank lines from appearing on Windows systems, and `writerows()` writes every row from the nested list in one call.

Exercise 65: Word Frequency from a File (Context manager)

Task: Read a text file and report the frequency of each word it contains.

```python
frequency = {}

with open("students.txt", "r") as file:
    for line in file:
        for word in line.strip().lower().split():
            frequency[word] = frequency.get(word, 0) + 1

for word, count in sorted(frequency.items()):
    print(f"{word}: {count}")
```

Explanation: Iterating over a file with `for line in file` reads it lazily, one line at a time, which is memory-efficient even for large files. Each line is then split into words and tallied using the same `.get(word, 0) + 1` counting pattern used earlier with strings, showing how the same logic applies whether the data comes from user input or a file.

MODULE 6 — Exception Handling

Exercise 66: Divide by Zero Handler (Basic try-except)

Task: Safely divide two numbers entered by the user, handling the case where the divisor is zero.

```python
try:
    a = float(input("Enter numerator: "))
    b = float(input("Enter denominator: "))
    result = a / b
    print(f"Result: {result}")
except ZeroDivisionError:
    print("Error: Cannot divide by zero.")
```

Explanation: Code that might fail is placed inside the `try` block. If a `ZeroDivisionError` occurs (dividing by zero), control jumps straight to the matching `except` block instead of crashing the program, and a friendly message is shown.

Exercise 67: Input Validation (Handling specific exceptions)

Task: Repeatedly ask for a number until valid numeric input is given, catching invalid (non-numeric) entries.

```python
while True:
    try:
        age = int(input("Enter your age: "))
        print(f"Your age is {age}")
        break
    except ValueError:
        print("That's not a valid number. Try again.")
```

Explanation: `int()` raises a `ValueError` if the input string isn't a valid integer (e.g., "abc"). Wrapping the conversion in a `try-except` inside a `while True` loop lets the program keep asking until the user provides input that actually converts successfully, at which point `break` exits the loop.

Exercise 68: Multiple Except Clauses

Task: Open a file and process a number from it, handling both a missing file and invalid content separately.

```python
```

```python
try:
    with open("data.txt", "r") as file:
        value = int(file.read().strip())
        print(f"Value: {value}")
except FileNotFoundError:
    print("Error: data.txt does not exist.")
except ValueError:
    print("Error: File does not contain a valid integer.")
```

Explanation: A single `try` block can be followed by several `except` clauses, each catching a different exception type. Python checks them in order and runs the first one that matches the exception actually raised, so a missing file and a bad value are reported with distinct, useful messages instead of one generic error.

Exercise 69: Guaranteed File Closing (finally clause)

Task: Demonstrate that a `finally` block runs whether or not an exception occurs, using it to guarantee a file is closed.

python

```python
file = None
try:
    file = open("students.txt", "r")
    content = file.read()
    print(content)
except FileNotFoundError:
    print("File not found.")
finally:
    if file:
        file.close()
        print("File closed.")
```

Explanation: The `finally` block always executes after `try`/`except`, regardless of whether an exception was raised or caught — making it the right place for cleanup code like closing a file. Checking `if file:` avoids an error in case `open()` itself failed and `file` was never assigned.

Exercise 70: Successful Division Message (try-except-else)

Task: Divide two numbers and print a success message only when no exception occurs, using an `else` clause.

python

```python
try:
    a = float(input("Enter numerator: "))
    b = float(input("Enter denominator: "))
    result = a / b
except ZeroDivisionError:
    print("Cannot divide by zero.")
else:
    print(f"Division successful! Result: {result}")
```

Explanation: The `else` clause runs only if the `try` block completed without raising any exception. This keeps "success path" code (like printing the result) separate from the risky code in `try`, making it clear which lines are expected to always succeed once the risky operation has passed.

Exercise 71: Custom Exception — Insufficient Balance

Task: Define a custom exception class and raise it when a bank withdrawal exceeds the account balance.

```python
class InsufficientBalanceError(Exception):
    def __init__(self, balance, amount):
        super().__init__(f"Cannot withdraw {amount}; balance is only {balance}"

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalanceError(balance, amount)
    return balance - amount

try:
    new_balance = withdraw(1000, 1500)
except InsufficientBalanceError as e:
    print(f"Transaction failed: {e}")
```

Explanation: `class InsufficientBalanceError(Exception)` creates a new exception type by inheriting from the built-in `Exception` class, with a custom message built in its `__init__`. `raise` triggers this exception manually when the business rule (insufficient funds) is violated, and it's caught just like any built-in exception.

Exercise 72: Custom Exception for Age Validation

Task: Raise a custom exception if an entered age is negative or unrealistically high.

```python
```

```python
class InvalidAgeError(Exception):
    pass

def validate_age(age):
    if age < 0 or age > 120:
        raise InvalidAgeError(f"{age} is not a valid age")
    return age

try:
    user_age = validate_age(int(input("Enter age: ")))
    print(f"Age accepted: {user_age}")
except InvalidAgeError as e:
    print(f"Invalid input: {e}")
```

Explanation: This custom exception uses `pass` in its body since it doesn't need extra behavior beyond what `Exception` already provides — the message passed to `raise InvalidAgeError(...)` is enough. Defining a specific exception class (rather than reusing a generic one) makes the error's meaning explicit both in the code and to anyone reading the traceback.

Exercise 73: Safe Matrix Element Access (Nested try-except)

Task: Safely access an element of a 2D list by row and column, handling both out-of-range indices and non-integer input.

python

```python
matrix = [[1, 2, 3], [4, 5, 6]]

try:
    row = int(input("Enter row index: "))
    try:
        col = int(input("Enter column index: "))
        print(f"Value: {matrix[row][col]}")
    except IndexError:
        print("Column index out of range.")
except (ValueError, IndexError):
    print("Invalid row index or input.")
```

Explanation: Nesting one `try-except` inside another allows different stages of the operation to be protected independently — here, the column access has its own `IndexError` handler distinct from the row's. The outer `except (ValueError, IndexError)` catches multiple exception types at once using a tuple.

Exercise 74: Password Strength Validator (Raising exceptions manually)

Task: Write a function that raises exceptions when a password fails length or character-variety requirements.

```python
def validate_password(password):
    if len(password) < 8:
        raise ValueError("Password must be at least 8 characters long")
    if not any(char.isdigit() for char in password):
        raise ValueError("Password must contain at least one digit")
    return True

try:
    pwd = input("Enter a password: ")
    validate_password(pwd)
    print("Password accepted.")
except ValueError as e:
    print(f"Rejected: {e}")
```

Explanation: `raise ValueError("...")` manually triggers an exception with a custom message when a rule is broken, even though nothing "crashed" in the usual sense — this is how a program enforces its own validation rules using the exception system. `any(char.isdigit() for char in password)` checks whether at least one character is a digit.

Exercise 75: Robust Number Input (Exception handling in a loop)

Task: Keep prompting the user until they enter a positive number, handling both non-numeric input and negative numbers.

```python
while True:
    try:
        value = float(input("Enter a positive number: "))
        if value <= 0:
            raise ValueError("Number must be positive")
        break
    except ValueError as e:
        print(f"Invalid input ({e}). Please try again.")

print(f"You entered: {value}")
```

Explanation: This combines two error sources in one loop: `float()` itself may raise `ValueError` for non-numeric text, and the program also manually `raise`s a `ValueError` for numbers that are zero or negative. Both are caught by the same `except ValueError` clause, and the loop only `break`s once a genuinely valid value is received.

MODULE 7 — OOP Concepts

Exercise 76: Car Class (Class & object basics)

Task: Create a `Car` class with attributes for make, model, and year, plus a method to display its details.

```python
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def display_info(self):
        print(f"{self.year} {self.make} {self.model}")

car1 = Car("Toyota", "Corolla", 2022)
car2 = Car("Honda", "Civic", 2023)

car1.display_info()
car2.display_info()
```

Explanation: `class Car:` defines a blueprint, and `__init__` is the constructor that runs automatically when a new `Car` object is created, storing the given values as `self.` attributes. Each object (`car1`, `car2`) keeps its own independent copy of these attributes, which is the essence of object-oriented programming.

Exercise 77: Student Class with Constructor

Task: Create a `Student` class that stores roll number, name, and marks, with a method to check pass/fail status.

```python
```

```python
class Student:
    def __init__(self, roll_no, name, marks):
        self.roll_no = roll_no
        self.name = name
        self.marks = marks

    def result(self):
        status = "Pass" if self.marks >= 40 else "Fail"
        print(f"{self.name} (Roll {self.roll_no}): {self.marks} marks - {status

s1 = Student(101, "Aarav", 76)
s2 = Student(102, "Diya", 32)

s1.result()
s2.result()
```

Explanation: The constructor `__init__(self, roll_no, name, marks)` requires three values at creation time, ensuring every `Student` object starts with complete data. The `result()` method then operates on `self.marks`, which belongs to that specific object, so each student's pass/fail check is independent.

Exercise 78: BankAccount Class (Instance methods)

Task: Create a `BankAccount` class with deposit and withdraw methods that update the balance.

```python
```

```python
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}. New balance: {self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient funds.")
        else:
            self.balance -= amount
            print(f"Withdrew {amount}. New balance: {self.balance}")

account = BankAccount("Riya", 500)
account.deposit(200)
account.withdraw(1000)
account.withdraw(300)
```

Explanation: `deposit` and `withdraw` are instance methods — they take `self` as their first parameter, giving them access to and the ability to modify that particular account's `balance`. Each method call like `account.deposit(200)` implicitly passes `account` as `self`.

Exercise 79: Object Counter (Class variables vs instance variables)

Task: Track how many `Employee` objects have been created using a class variable, while each object keeps its own name as an instance variable.

python

```python
class Employee:
    count = 0  # class variable, shared by all objects

    def __init__(self, name):
        self.name = name          # instance variable, unique per object
        Employee.count += 1

e1 = Employee("Sam")
e2 = Employee("Priya")
e3 = Employee("Kabir")

print(f"Total employees created: {Employee.count}")
print(f"e1's name: {e1.name}")
```

Explanation: `count` is defined directly inside the class body, so it's shared across every instance — incrementing `Employee.count` in `__init__` affects the same value for all objects. `self.name`, in contrast, is set individually per object, so `e1.name` and `e2.name` hold different values even though both objects share the same class.

Exercise 80: Vehicle Inheritance Hierarchy

Task: Create a base `Vehicle` class and derive `Car` and `Bike` classes from it, adding type-specific behavior.

python

```python
class Vehicle:
    def __init__(self, brand, speed):
        self.brand = brand
        self.speed = speed

    def describe(self):
        print(f"{self.brand} moves at {self.speed} km/h")

class Car(Vehicle):
    def honk(self):
        print(f"{self.brand} car says: Beep beep!")

class Bike(Vehicle):
    def ring_bell(self):
        print(f"{self.brand} bike says: Ring ring!")

car = Car("Toyota", 180)
bike = Bike("Hero", 40)

car.describe()
car.honk()
bike.describe()
bike.ring_bell()
```

Explanation: `class Car(Vehicle):` makes `Car` a subclass of `Vehicle`, automatically inheriting its `__init__` and `describe()` method without rewriting them. Each subclass then adds its own unique method (`honk` or `ring_bell`), showing how inheritance lets shared behavior live in one place while specialized behavior stays in the subclasses.

Exercise 81: Multiple Inheritance

Task: Create two parent classes and a child class that inherits from both, demonstrating multiple inheritance.

python

```python
class Swimmer:
    def swim(self):
        print("Can swim")

class Runner:
    def run(self):
        print("Can run")

class Triathlete(Swimmer, Runner):
    def bike(self):
        print("Can bike")

athlete = Triathlete()
athlete.swim()
athlete.run()
athlete.bike()

print(Triathlete.__mro__)
```

Explanation: `class Triathlete(Swimmer, Runner):` inherits methods from both parent classes at once, so a `Triathlete` object can call `swim()`, `run()`, and its own `bike()`. `__mro__` (Method Resolution Order) shows the order Python searches through parent classes when looking up a method — useful when multiple parents might define the same method name.

Exercise 82: Shape Area with Method Overriding

Task: Create a `Shape` base class with a generic `area()` method, then override it in `Circle` and `Square` subclasses.

python

```python
class Shape:
    def area(self):
        return 0

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

shapes = [Circle(5), Square(4)]
for shape in shapes:
    print(f"{type(shape).__name__} area: {shape.area():.2f}")
```

Explanation: Both `Circle` and `Square` provide their own `area()` method, which **overrides** the generic version defined in `Shape`. When `shape.area()` is called inside the loop, Python uses each object's own overridden version rather than the parent's, even though all objects are handled through the same loop.

Exercise 83: Calculator with Simulated Overloading

Task: Simulate method overloading (which Python doesn't support natively) using default arguments, so `add()` works with two or three numbers.

```python
class Calculator:
    def add(self, a, b, c=0):
        return a + b + c

calc = Calculator()
print(calc.add(2, 3))
print(calc.add(2, 3, 4))
```

Explanation: Python does not allow defining multiple methods with the same name but different parameter counts, unlike some other languages. Instead, a single method with a default parameter (`c=0`) can behave as if it were "overloaded," accepting either two or three

arguments depending on the call.

Exercise 84: Vector Addition (Operator overloading)

Task: Create a `Vector` class and overload the `+` operator so two vectors can be added directly.

python

```python
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

v1 = Vector(2, 3)
v2 = Vector(4, 1)
v3 = v1 + v2

print(f"Result: {v3}")
```

Explanation: Defining `__add__(self, other)` tells Python what `+` should do when used between two `Vector` objects — this is operator overloading. Writing `v1 + v2` is internally translated to `v1.__add__(v2)`, returning a brand-new `Vector` combining both.

Exercise 85: Encapsulated BankAccount (Data hiding)

Task: Protect a `BankAccount`'s balance from direct external modification using a private attribute.

python

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance  # private attribute (name-mangled)

    def get_balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

account = BankAccount("Diya", 1000)
account.deposit(500)
print(f"Balance: {account.get_balance()}")

# Direct access fails / doesn't behave as expected:
# print(account.__balance)  # AttributeError
```

Explanation: Prefixing an attribute with double underscores (`__balance`) triggers Python's name mangling, making it much harder to access accidentally from outside the class (it becomes `_BankAccount__balance` internally). This enforces **data hiding**: outside code must go through controlled methods like `get_balance()` and `deposit()` rather than modifying `__balance` directly.

Exercise 86: Validated Attribute with @property

Task: Use the `@property` decorator to create a getter and setter for an `age` attribute that rejects negative values.

```python
```

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self._age = age

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError("Age cannot be negative")
        self._age = value

p = Person("Kabir", 25)
print(p.age)

p.age = 30       # uses the setter
print(p.age)

p.age = -5        # raises ValueError
```

Explanation: `@property` lets `age` be accessed like a plain attribute (`p.age`) while actually running a method behind the scenes. The paired `@age.setter` intercepts assignments (`p.age = 30`) and can validate the new value before storing it in `_age`, combining the convenience of attribute syntax with the safety of a method.

Exercise 87: Extending a Constructor with super()

Task: Create a `Person` base class and an `Employee` subclass that extends the constructor using `super()`.

```python
```

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Employee(Person):
    def __init__(self, name, age, salary):
        super().__init__(name, age)
        self.salary = salary

    def display(self):
        print(f"{self.name}, {self.age} years old, earns {self.salary}")

emp = Employee("Riya", 28, 60000)
emp.display()
```

Explanation: `super().__init__(name, age)` calls the parent class's constructor to handle the attributes `Person` already knows how to set up, so `Employee` doesn't need to repeat that logic. `Employee` then adds its own extra attribute, `salary`, building on top of what the parent provides.

Exercise 88: Class Method and Static Method

Task: Create an `Employee` class with a class method that tracks the total number of employees and a static method that validates a salary.

```
python
```

```python
class Employee:
    total_employees = 0

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
        Employee.total_employees += 1

    @classmethod
    def get_total(cls):
        return cls.total_employees

    @staticmethod
    def is_valid_salary(salary):
        return salary > 0

e1 = Employee("Sam", 45000)
e2 = Employee("Priya", 52000)

print(f"Total employees: {Employee.get_total()}")
print(f"Is -5000 a valid salary? {Employee.is_valid_salary(-5000)}")
```

Explanation: A `@classmethod` receives the class itself (`cls`) instead of an instance, making it suitable for operations on class-level data like `total_employees`. A `@staticmethod` receives neither `self` nor `cls` — it's essentially a regular function grouped inside the class for organizational purposes, used here for a validation check unrelated to any specific object's state.

Exercise 89: Polymorphism with Animal Sounds

Task: Demonstrate polymorphism by giving different animal classes the same method name, each with different behavior.

```python
```

```python
class Animal:
    def sound(self):
        raise NotImplementedError("Subclass must implement this method")

class Dog(Animal):
    def sound(self):
        return "Woof!"

class Cat(Animal):
    def sound(self):
        return "Meow!"

class Cow(Animal):
    def sound(self):
        return "Moo!"

animals = [Dog(), Cat(), Cow()]
for animal in animals:
    print(f"{type(animal).__name__}: {animal.sound()}")
```

Explanation: All three subclasses implement a `sound()` method with the same name but different behavior. The loop calls `animal.sound()` without needing to know which specific subclass each object belongs to — Python automatically calls the correct version at runtime, which is the essence of polymorphism.

Exercise 90: Custom str and repr

Task: Give a `Book` class human-readable (`__str__`) and developer-oriented (`__repr__`) string representations.

```python
```

```python
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        return f"'{self.title}' by {self.author}"

    def __repr__(self):
        return f"Book(title={self.title!r}, author={self.author!r})"

book = Book("The Alchemist", "Paulo Coelho")
print(str(book))    # uses __str__
print(repr(book))   # uses __repr__
print(book)          # print() uses __str__ by default
```

Explanation: `__str__` defines a friendly display used by `print()` and `str()`, aimed at end users, while `__repr__` defines an unambiguous representation aimed at developers (often resembling the code needed to recreate the object) and is used by `repr()` and in interactive shells. Defining both gives a class appropriate output for both contexts.

MODULE 8 — Database (SQL with Python)

These exercises use Python's built-in `sqlite3` module, which requires no separate database server install — ideal for learning SQL fundamentals and CRUD operations directly from Python.

Exercise 91: Connect and Create a Table

Task: Connect to an SQLite database (creating it if it doesn't exist) and create a `students` table.

```python
```

```python
import sqlite3

conn = sqlite3.connect("academy.db")
cursor = conn.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS students (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        age INTEGER,
        course TEXT
    )
""")

conn.commit()
conn.close()
print("Table created successfully.")
```

Explanation: `sqlite3.connect("academy.db")` opens a connection to a database file, creating it if it doesn't already exist. A `cursor` is used to execute SQL statements; `IF NOT EXISTS` prevents an error if the table is already there, and `conn.commit()` saves the change permanently to the file.

Exercise 92: Insert a Single Record

Task: Insert one new student record into the `students` table.

```python
import sqlite3

conn = sqlite3.connect("academy.db")
cursor = conn.cursor()

cursor.execute("INSERT INTO students (name, age, course) VALUES (?, ?, ?)",
               ("Aarav", 20, "Python"))

conn.commit()
print(f"Record inserted with id {cursor.lastrowid}")
conn.close()
```

Explanation: The `?` placeholders in the SQL string are filled in from the tuple passed as the second argument to `execute()` — this is a **parameterized query**, which safely inserts values without building the SQL string manually. `cursor.lastrowid` reports the auto-generated ID of the newly inserted row.

Exercise 93: Insert Multiple Records (executemany)

Task: Insert several student records at once using `executemany()`.

```python
import sqlite3

conn = sqlite3.connect("academy.db")
cursor = conn.cursor()

students = [
    ("Diya", 21, "Data Analytics"),
    ("Kabir", 19, "Cyber Security"),
    ("Riya", 22, "AI/ML"),
]

cursor.executemany("INSERT INTO students (name, age, course) VALUES (?, ?, ?)",

conn.commit()
print(f"{cursor.rowcount} records inserted.")
conn.close()
```

Explanation: `executemany()` runs the same parameterized SQL statement once for every tuple in the `students` list, avoiding a manual loop with repeated `execute()` calls. `cursor.rowcount` reports how many rows were affected by the last operation.

Exercise 94: Select All Records (Read)

Task: Retrieve and display every record from the `students` table.

```python
import sqlite3

conn = sqlite3.connect("academy.db")
cursor = conn.cursor()

cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()

for row in rows:
    print(row)

conn.close()
```

Explanation: `SELECT * FROM students` requests every column of every row. `cursor.fetchall()` retrieves all matching rows as a list of tuples, which can then be looped over and printed — this is the "Read" operation in CRUD.

Exercise 95: Select with WHERE Clause (Filtering)

Task: Retrieve only the students enrolled in a specific course.

python

```python
import sqlite3

conn = sqlite3.connect("academy.db")
cursor = conn.cursor()

course = input("Enter course to search: ")
cursor.execute("SELECT name, age FROM students WHERE course = ?", (course,))
results = cursor.fetchall()

if results:
    for name, age in results:
        print(f"{name} ({age} years old)")
else:
    print("No students found for that course.")

conn.close()
```

Explanation: The `WHERE course = ?` clause filters rows so only matching records are returned, rather than fetching everything and filtering in Python — this is more efficient, especially for large tables. Note the trailing comma in `(course,)`, which is required to make it a one-element tuple rather than just a plain value in parentheses.

Exercise 96: Update a Record

Task: Update a student's course based on their name.

python

```python
import sqlite3

conn = sqlite3.connect("academy.db")
cursor = conn.cursor()

name = input("Enter student name to update: ")
new_course = input("Enter new course: ")

cursor.execute("UPDATE students SET course = ? WHERE name = ?", (new_course, na
conn.commit()

print(f"{cursor.rowcount} record(s) updated.")
conn.close()
```

Explanation: `UPDATE students SET course = ? WHERE name = ?` changes only the rows that match the `WHERE` condition — omitting `WHERE` would update every row in the table, so it's essential for a targeted change. `conn.commit()` is required after any write operation (`INSERT`, `UPDATE`, `DELETE`) for the change to be saved to disk.

Exercise 97: Delete a Record

Task: Delete a student record from the table by name, with a confirmation step.

python

```python
import sqlite3

conn = sqlite3.connect("academy.db")
cursor = conn.cursor()

name = input("Enter student name to delete: ")
confirm = input(f"Are you sure you want to delete '{name}'? (yes/no): ")

if confirm.lower() == "yes":
    cursor.execute("DELETE FROM students WHERE name = ?", (name,))
    conn.commit()
    print(f"{cursor.rowcount} record(s) deleted.")
else:
    print("Deletion cancelled.")

conn.close()
```

Explanation: As with `UPDATE`, the `WHERE` clause in `DELETE FROM students WHERE name = ?` ensures only the targeted row is removed. Adding a confirmation prompt before executing the delete is a practical safeguard against accidental data loss, since `DELETE` cannot be

undone once committed.

Exercise 98: Full CRUD Menu Program

Task: Combine Create, Read, Update, and Delete into a single menu-driven program.

```
python
```

```python
import sqlite3

def connect():
    return sqlite3.connect("academy.db")

def add_student(name, age, course):
    conn = connect()
    conn.execute("INSERT INTO students (name, age, course) VALUES (?, ?, ?)", (
    conn.commit()
    conn.close()

def view_students():
    conn = connect()
    for row in conn.execute("SELECT * FROM students"):
        print(row)
    conn.close()

def update_course(name, new_course):
    conn = connect()
    conn.execute("UPDATE students SET course = ? WHERE name = ?", (new_course,
    conn.commit()
    conn.close()

def delete_student(name):
    conn = connect()
    conn.execute("DELETE FROM students WHERE name = ?", (name,))
    conn.commit()
    conn.close()

while True:
    print("\n1. Add  2. View  3. Update Course  4. Delete  5. Exit")
    choice = input("Choose an option: ")

    if choice == "1":
        add_student(input("Name: "), int(input("Age: ")), input("Course: "))
    elif choice == "2":
        view_students()
    elif choice == "3":
        update_course(input("Name to update: "), input("New course: "))
    elif choice == "4":
        delete_student(input("Name to delete: "))
    elif choice == "5":
        break
    else:
        print("Invalid choice.")
```

Explanation: Each CRUD operation is wrapped in its own function with a fresh `connect()`/`close()` cycle, keeping the menu loop simple and each database operation self-contained and reusable. This structure mirrors how database logic is organized in larger, real-world applications.

Exercise 99: Parameterized Queries vs. SQL Injection

Task: Demonstrate why parameterized queries are used by comparing safe and unsafe query construction.

python

```python
import sqlite3

conn = sqlite3.connect("academy.db")
cursor = conn.cursor()

user_input = "Aarav' OR '1'='1"

# UNSAFE (do not do this) - vulnerable to SQL injection:
# query = f"SELECT * FROM students WHERE name = '{user_input}'"
# cursor.execute(query)

# SAFE - parameterized query:
cursor.execute("SELECT * FROM students WHERE name = ?", (user_input,))
results = cursor.fetchall()

print(f"Matching records: {len(results)}")
conn.close()
```

Explanation: Building SQL by directly inserting user input into a string (the commented-out unsafe version) lets malicious input like `"' OR '1'='1"` change the meaning of the query and potentially return every row. The parameterized version treats `user_input` strictly as a data value, never as part of the SQL syntax, which is why `?` placeholders should always be used with untrusted input.

Exercise 100: Aggregate Functions (COUNT and AVG)

Task: Use SQL aggregate functions to report the total number of students and their average age.

python

```python
import sqlite3

conn = sqlite3.connect("academy.db")
cursor = conn.cursor()

cursor.execute("SELECT COUNT(*), AVG(age) FROM students")
total, avg_age = cursor.fetchone()

print(f"Total students: {total}")
print(f"Average age: {avg_age:.1f}" if avg_age else "No data available")

cursor.execute("SELECT course, COUNT(*) FROM students GROUP BY course")
print("\nStudents per course:")
for course, count in cursor.fetchall():
    print(f"  {course}: {count}")

conn.close()
```

Explanation: `COUNT(*)` and `AVG(age)` are SQL aggregate functions computed directly by the database engine, which is far more efficient than pulling all rows into Python and calculating manually. `GROUP BY course` further splits the count per distinct course value, a common pattern for summarizing data.

Summary

Module	Range	Count
Core Python Concepts	1–15	15
Collections	16–35	20
Functions	36–47	12
Modules	48–55	8
Input/Output & Files	56–65	10
Exception Handling	66–75	10
OOP Concepts	76–90	15
Database (SQL)	91–100	10
Total		**100**